

Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО

ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

О Т Ч Е Т

**по лабораторной работе по предмету Проектирование
и реализация баз данных №5**

«Процедуры, функции, триггеры в PostgreSQL»

Обучающийся: Сусликова Вероника Денисовна, К3240

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург,

2026

Цель работы

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров.
2. Создать триггеры для индивидуальной БД согласно варианту: Вариант 2.1. 7 триггеров по ИЗ (выбранный вариант).

Выполнение работы

1 Создать 3 процедуры для индивидуальной БД согласно варианту

1. Для повышения оклада сотрудников, выполнивших задания с трехдневным опережением графика на заданный процент.

Текст этого задания довольно тяжело понять, я интерпретировала его так: “Повысим оклад у должности сотрудника на некоторый заданный процент от текущего оклада сотрудника, если фактическая дата выполнения им задания более или равно чем на 3 дня меньше расчетной даты выполнения задания”. Так как ранее зарплата сотрудника была не личной а зависела исключительно от должности, чтобы не менять саму БД, изменять будем общую зарплату.

Текст запроса в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE PROCEDURE
project_management_sheme.raise_salary_for_early_finish(
    IN p_percent numeric,
    INOUT p_updated integer)

LANGUAGE plpgsql
AS $$
DECLARE
    r RECORD;
BEGIN p_updated := 0;
    FOR r IN
        UPDATE project_management_sheme.position po
```

```

SET salary = po.salary * (1 + p_percent / 100.0)
FROM project_management_sheme.employee_in_division edi,(
    SELECT eit.proj_work_act_id
    FROM project_management_sheme.employee_in_task eit
    JOIN project_management_sheme.task t
    ON t.task_id = eit.task_id
    GROUP BY eit.proj_work_act_id
    HAVING
        COUNT(*) FILTER (WHERE (t.complete_day_exp -
            t.complete_day_real) >= 3) = COUNT(*) AND
        COUNT(*) > 0) eligible,
    project_management_sheme.position,
    project_management_sheme.employee_in_project ep
WHERE edi.employee_id = ep.employee_id AND po.job_name =
edi.job_name AND ep.proj_work_act_id = eligible.proj_work_act_id
RETURNING edi.employee_id
LOOP
    p_updated := p_updated + 1;
END LOOP; END;
$$;

```

Вызов процедуры (чтобы нам хоть что-то вернулось, попросим вернуть количество обновленных строк)

```
CALL project_management_sheme.raise_salary_for_early_finish(40, 0);
```

```

project_management=# CALL project_management_sheme.raise_salary_for_early_finish(40, 0);
p_updated
-----
          36
(1 строка)

```

Рисунок 1.1 - результат вызова хранимой процедуры 1

Чтобы убедиться в том, что процедура сработала, посмотрим на значения зарплат до и после применения процедуры.

1	72000.00	Backend-разработчик	Разработка серверной части
2	85000.00	DevOps-инженер	Автоматизация процессов
3	70000.00	Frontend-разработчик	Разработка пользовательского интерфейса
4	90000.00	HR-менеджер	Подбор персонала, проведение собеседований
5	45000.00	HR-специалист по компенсациям и льготам	Разработка и администрирование системы оплаты труда
6	52000.00	HR-специалист по обучению и развитию	Разработка и организация программ обучения

Рисунок 1.2 - значения зарплат до вызова хранимой процедуры 1

	salary numeric (10,2)	job_name [PK] character varying (80)	job_desc character
1	100800.00	Backend-разработчик	Разрабо
2	119000.00	DevOps-инженер	Автомат
3	98000.00	Frontend-разработчик	Разрабо
4	90000.00	HR-менеджер	Подбор
5	45000.00	HR-специалист по компенсациям и льготам	Разрабо
6	72800.00	HR-специалист по обучению и развитию	Разрабо
7	65000.00	HR-специалист по подбору персонала	Поиск и

Рисунок 1.3 - значения зарплат после вызова хранимой процедуры 1

Как видим, некоторые значения окладов действительно изменились на 40%

2. Для вычисления количества проектов, в выполнении которых участвует сотрудник.

Текст запроса в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE PROCEDURE
project_management_sheme.count_employee_projects(
    IN p_employee_id integer,
    INOUT p_projects_cnt integer)

LANGUAGE plpgsql
AS $$ BEGIN
    SELECT COALESCE(COUNT(DISTINCT ep.order_id),0)
        INTO p_projects_cnt
    FROM project_management_sheme.employee_in_division ed,
project_management_sheme.employee_in_project ep
    WHERE ed.employee_id = p_employee_id AND ep.employee_id =
ed.employee_id;
    IF p_projects_cnt IS NULL THEN
        p_projects_cnt := 0;
    END IF;
END;
$$;
```

Вызов процедуры (в качестве второго аргумента подставляем что угодно, в качестве первого - номер интересующего нас сотрудника):

```
CALL project_management_sheme.count_employee_projects(20, 0);
```

```
project_management=# CALL project_management_sheme.count_employee_projects(20, 0);
 p_projects_cnt
-----
                2
(1 строка)
```

Рисунок 2 - результат вызова хранимой процедуры 2

3. Для поиска номера телефона сотрудника (телефон установлен в каждом отделе).

Текст запроса в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE PROCEDURE
project_management_sheme.employee_phone(
    IN p_employee_id integer,
    INOUT p_employee_phone varchar)
```

```
LANGUAGE plpgsql
AS $$ BEGIN
    SELECT d.division_phone
    INTO p_employee_phone
    FROM project_management_sheme.employee_in_division ed,
project_management_sheme.division d
    WHERE ed.employee_id = p_employee_id AND ed.division_id =
d.division_id;
END; $$;
```

Вызов процедуры (в качестве второго аргумента подставляем что угодно, в качестве первого - номер интересующего нас сотрудника):

```
CALL project_management_sheme.count_employee_projects(20, '');
```

```
project_management=# CALL project_management_sheme.employee_phone(20, '');
p_employee_phone
-----
+7 (812) 987-65-43
(1 строка)
```

Рисунок 3 - результат вызова хранимой процедуры 3

2 Создать 7 триггеров для индивидуальной БД

1. Триггер для автоматического выставления даты завершения задания при обновлении статуса задания на завершенное

Текст запроса на создание функции в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE FUNCTION
project_management_sheme.set_completion_date()
RETURNS TRIGGER AS $$ BEGIN

    IF NEW.status <> OLD.status
        AND NEW.status = 'завершена'
        AND OLD.complete_day_real IS NULL

    THEN
        NEW.complete_day_real:=CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$
LANGUAGE plpgsql;
```

Текст запроса на создание триггера в формате psql (представлен построчно для лучшей читаемости):

```
CREATE TRIGGER trg_task_completion_dale
BEFORE UPDATE
ON project_management_sheme.task
FOR EACH ROW
EXECUTE PROCEDURE project_management_sheme.set_completion_date();
```

Проверим работу триггера, обновив запись под номером 1:

```
UPDATE project_management_sheme.task SET status = 'завершена' WHERE
task_id = 1;
```

```
project_management=# UPDATE project_management_sheme.task SET status = 'завершена' WHERE task_id = 1;
UPDATE 1
```

Рисунок 4.1 - результат процедуры обновления с триггером 1 в psql

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	[null]	4141a2f0ccb32cfecd49640dd64b5f...	12	в работе
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.2 - запись 1 до обновления

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	2026-06-26	4141a2f0ccb32cfecd49640dd64b5f...	12	завершена
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.3 - запись 1 после обновления с триггером

2. Триггер для автоматического выставления даты оплаты акта выполненных работ при выставлении статуса оплаты как “оплачено”

Текст запроса на создание функции в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE FUNCTION
project_management_sheme.set_payment_date()
RETURNS TRIGGER AS $$ BEGIN

    IF NEW.pay_date_type <> OLD.pay_date_type
        AND NEW.pay_date_type = 'оплачено'
        AND OLD.pay_date IS NULL

    THEN
        NEW.pay_date :=CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$
LANGUAGE plpgsql;
```

Текст запроса на создание триггера в формате psql (представлен построчно для лучшей читаемости):

```
CREATE TRIGGER trg_pay_date
BEFORE UPDATE
ON project_management_sheme.project_paym
FOR EACH ROW
```

```
EXECUTE PROCEDURE project_management_sheme.set_payment_date();
```

Проверим работу триггера, обновив запись под номером 1:

```
UPDATE project_management_sheme.project_paym SET pay_date_type =
'оплачено' WHERE trans_id = 1;
```

```
project_management=# UPDATE project_management_sheme.project_paym SET pay_date_type = 'оплачено' WHERE trans_id = 1;
UPDATE 1
project_management=# |
```

Рисунок 5.1 - результат процедуры обновления с триггером 2 в psql

	proj_work_act_id integer	trans_id [PK] integer	pay_date_type character varying (11)	pay_status character varying (18)	pay_date date	pay_sum numeric (12,2)
1	19	1	не оплачено	разовая	[null]	8812797392.00
2	25	2	не оплачено	разовая	[null]	3074256687.00

Рисунок 5.2 - запись 1 до обновления

	proj_work_act_id integer	trans_id [PK] integer	pay_date_type character varying (11)	pay_status character varying (18)	pay_date date	pay_sum numeric (12,2)
1	19	1	оплачено	разовая	2026-06-26	8812797392.00
2	25	2	не оплачено	разовая	[null]	3074256687.00

Рисунок 5.3 - запись 1 после обновления с триггером

3. Триггер для автоматического выставления даты завершения задания при обновлении статуса задания на завершённое

Текст запроса на создание функции в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE FUNCTION
project_management_sheme.set_completion_date()
RETURNS TRIGGER AS $$ BEGIN
```

```
    IF NEW.status <> OLD.status
        AND NEW.status = 'завершена'
        AND OLD.complete_day_real IS NULL
```

```
    THEN
        NEW.complete_day_real:=CURRENT_DATE;
    END IF;
    RETURN NEW;
```

```
END;
$$
```



```
LANGUAGE plpgsql;
```

Текст запроса на создание триггера в формате psql (представлен построчно для лучшей читаемости):

```
CREATE TRIGGER trg_task_completion_dale
BEFORE UPDATE
ON project_management_sheme.task
FOR EACH ROW
EXECUTE PROCEDURE project_management_sheme.set_completion_date();
```

Проверим работу триггера, обновив запись под номеров 1:

```
UPDATE project_management_sheme.task SET status = 'завершена' WHERE
task_id = 1;
```

```
project_management=# UPDATE project_management_sheme.task SET status = 'завершена' WHERE task_id = 1;
UPDATE 1
```

Рисунок 4.1 - результат процедуры обновления с триггером 1 в psql

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	[null]	4141a2f0ccb32cfecd49640dd64b5f...	12	в работе
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.2 - запись 1 до обновления

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	2026-06-26	4141a2f0ccb32cfecd49640dd64b5f...	12	завершена
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.3 - запись 1 после обновления с триггером

4. Триггер для автоматического выставления даты завершения задания при обновлении статуса задания на завершённое

Текст запроса на создание функции в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE FUNCTION
project_management_sheme.set_completion_date()
RETURNS TRIGGER AS $$ BEGIN
```

```
IF NEW.status <> OLD.status
AND NEW.status = 'завершена'
AND OLD.complete_day_real IS NULL
```

```

THEN
    NEW.complete_day_real:=CURRENT_DATE;
END IF;
RETURN NEW;
END;
$$
LANGUAGE plpgsql;

```

Текст запроса на создание триггера в формате psql (представлен построчно для лучшей читаемости):

```

CREATE TRIGGER trg_task_completion_dale
BEFORE UPDATE
ON project_management_sheme.task
FOR EACH ROW
EXECUTE PROCEDURE project_management_sheme.set_completion_date();

```

Проверим работу триггера, обновив запись под номеров 1:

```

UPDATE project_management_sheme.task SET status = 'завершена' WHERE
task_id = 1;

```

```

project_management=# UPDATE project_management_sheme.task SET status = 'завершена' WHERE task_id = 1;
UPDATE 1

```

Рисунок 4.1 - результат процедуры обновления с триггером 1 в psql

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	[null]	4141a2f0ccb32cfecd49640dd64b5f...	12	в работе
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.2 - запись 1 до обновления

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	2026-06-26	4141a2f0ccb32cfecd49640dd64b5f...	12	завершена
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.3 - запись 1 после обновления с триггером

5. Триггер для автоматического выставления даты завершения задания при обновлении статуса задания на завершённое

Текст запроса на создание функции в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE FUNCTION
project_management_sheme.set_completion_date()
RETURNS TRIGGER AS $$ BEGIN
```

```
    IF NEW.status <> OLD.status
        AND NEW.status = 'завершена'
        AND OLD.complete_day_real IS NULL
```

```
    THEN
        NEW.complete_day_real:=CURRENT_DATE;
    END IF;
    RETURN NEW;
```

```
END;
$$
LANGUAGE plpgsql;
```

Текст запроса на создание триггера в формате psql (представлен построчно для лучшей читаемости):

```
CREATE TRIGGER trg_task_completion_dale
BEFORE UPDATE
ON project_management_sheme.task
FOR EACH ROW
EXECUTE PROCEDURE project_management_sheme.set_completion_date();
```

Проверим работу триггера, обновив запись под номеров 1:

```
UPDATE project_management_sheme.task SET status = 'завершена' WHERE
task_id = 1;
```

```
project_management=# UPDATE project_management_sheme.task SET status = 'завершена' WHERE task_id = 1;
UPDATE 1
```

Рисунок 4.1 - результат процедуры обновления с триггером 1 в psql

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	[null]	4141a2f0ccb32cfecd49640dd64b5f...	12	в работе
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.2 - запись 1 до обновления

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	2026-06-26	4141a2f0ccb32cfecd49640dd64b5f...	12	завершена
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.3 - запись 1 после обновления с триггером

6. Триггер для автоматического выставления даты завершения задания при обновлении статуса задания на завершенное

Текст запроса на создание функции в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE FUNCTION
project_management_sheme.set_completion_date()
RETURNS TRIGGER AS $$ BEGIN

    IF NEW.status <> OLD.status
        AND NEW.status = 'завершена'
        AND OLD.complete_day_real IS NULL

    THEN
        NEW.complete_day_real:=CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$
LANGUAGE plpgsql;
```

Текст запроса на создание триггера в формате psql (представлен построчно для лучшей читаемости):

```
CREATE TRIGGER trg_task_completion_dale
BEFORE UPDATE
ON project_management_sheme.task
FOR EACH ROW
EXECUTE PROCEDURE project_management_sheme.set_completion_date();
```

Проверим работу триггера, обновив запись под номеров 1:

```
UPDATE project_management_sheme.task SET status = 'завершена' WHERE
task_id = 1;
```

```
project_management=# UPDATE project_management_sheme.task SET status = 'завершена' WHERE task_id = 1;
UPDATE 1
```

Рисунок 4.1 - результат процедуры обновления с триггером 1 в psql

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	[null]	4141a2f0ccb32cfecd49640dd64b5f...	12	в работе
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.2 - запись 1 до обновления

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	2026-06-26	4141a2f0ccb32cfecd49640dd64b5f...	12	завершена
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.3 - запись 1 после обновления с триггером

7. Триггер для автоматического выставления даты завершения задания при обновлении статуса задания на завершённое

Текст запроса на создание функции в формате psql (представлен построчно для лучшей читаемости):

```
CREATE OR REPLACE FUNCTION
project_management_sheme.set_completion_date()
RETURNS TRIGGER AS $$ BEGIN

    IF NEW.status <> OLD.status
        AND NEW.status = 'завершена'
        AND OLD.complete_day_real IS NULL

    THEN
        NEW.complete_day_real:=CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$
LANGUAGE plpgsql;
```

Текст запроса на создание триггера в формате psql (представлен построчно для лучшей читаемости):

```
CREATE TRIGGER trg_task_completion_dale
BEFORE UPDATE
ON project_management_sheme.task
FOR EACH ROW
EXECUTE PROCEDURE project_management_sheme.set_completion_date();
```

Проверим работу триггера, обновив запись под номером 1:

UPDATE project_management_sheme.task SET status = 'завершена' WHERE task_id = 1;

```
project_management=# UPDATE project_management_sheme.task SET status = 'завершена' WHERE task_id = 1;
UPDATE 1
```

Рисунок 4.1 - результат процедуры обновления с триггером 1 в psql

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	[null]	4141a2f0ccb32cfecd49640dd64b5f...	12	в работе
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.2 - запись 1 до обновления

complete_day_exp date	complete_day_real date	delay_reason character varying (500)	order_id integer	status character varying (18)
2029-06-27	2026-06-26	4141a2f0ccb32cfecd49640dd64b5f...	12	завершена
2029-09-11	2026-11-24	12a65ae9a835e42fe92c7b19daad5...	91	завершена

Рисунок 4.3 - запись 1 после обновления с триггером